Table des matières

Labo 08 — Lire des séries temporelles 1
Avant de commencer — la base de séries temporelles 2
Étape 1 — Le référentiel : de l'étiquette à l'équipement 3
Étape 2 — La GMAO, et ses dates qui n'en sont pas 4
Étape 3 — Le compteur électrique : un horodatage qui dit la fin 5
Étape 4 — L'historien au bronze : ce qui n'est pas une mesure 6
Étape 5 — Les deux jours où l'horloge ment 8
Étape 6 — La qualité, les coupures, et le capteur qui ne bouge plus 9
Étape 7 — L'incident du compresseur 2, lu dans les points 12
Étape 8 — Rassembler le diagnostic, et le versionner 14
Étape 9 — Résoudre l'heure, et le prouver 14
Étape 10 — L'argent : découper, pondérer, écrire 15
Étape 11 — Ce que la pondération change, mesuré 16
Étape 12 — Ce que l'argent livre, et à qui 17
Étape 13 — Charger la base de séries temporelles 17
Étape 14 — L'agrégat continu, et la preuve 18
Récapitulatif 19
Dépannage 20

Labo 08 — Lire des séries temporelles

Labo · Cours 8/12 — Formation Data Engineering

Ce que vous allez faire. Interroger les quatre fichiers que l'usine de Val-des-Aulnes vous livre, et répondre à une seule question : à quoi peut-on se fier. Vous compterez les lignes qui ne sont pas des mesures, les points sans valeur, les heures qui se répètent et celles qui n'existent pas, et vous distinguerez un capteur mort d'un équipement arrêté. Vous finirez sur l'incident du compresseur 2, que la GMAO date du 28 octobre 2025 et que l'historien annonçait treize jours plus tôt. Objectif de fin : chaque piège compté, avec son chiffre, et un script de diagnostic qui les rejoue tous.

© 2026 iAlexMG. Tous droits réservés.

Le cours compte deux séances. Celle-ci va jusqu'à l'étape 8, et elle se tient seule : elle mesure le terrain. La seconde nettoie ces séries, les rééchantillonne à l'heure et les charge dans une base de séries temporelles.

Deux terminaux servent dans cette séance. L'Anaconda Prompt, environnement `formations` actif, pour Git et pour créer les fichiers. La ligne de commande DuckDB, lancée depuis `Data_Domain\Projet_Boreal`, pour tout le SQL : les chemins des requêtes sont relatifs à ce dossier. Ubuntu et Docker n'entrent qu'à la seconde séance, pour la base de séries temporelles, et le bloc qui suit est le seul de la première à les ouvrir.

La liste de contrôle de la séance :

- ☐ Le référentiel lu : 23 lignes d'actifs, 6 zones, 17 équipements, 58 étiquettes
- ☐ La GMAO lue : 825 bons, 444 préventifs et 381 correctifs, 4 encore ouverts, et des dates qui se trient de travers
- ☐ Le compteur lu : 148 891 relevés, 247 estimés, une pointe à 1 906 kW, et 27 116 relevés qui changent de jour selon le fuseau
- ☐ L'historien compté : 550 087 lignes, dont 10 556 valeurs de départ et 10 525 points d'archivage de minuit
- ☐ Les deux changements d'heure vus : 49 paires en double le 2025-11-02, aucun point entre 02:00 et 03:00 le 2026-03-08
- ☐ La qualité comptée : 538 938 bonnes, 64 douteuses, 529 mauvaises et sans valeur, sur 44 jours de coupure
- ☐ Le capteur figé trouvé : l'humidité de la cabine, 49,8 heures à 54,52 %, et les faux positifs des équipements arrêtés écartés, y compris le compresseur 1 qui s'arrête à 0,01
- ☐ L'incident lu dans les points : la vibration du compresseur 2 passe de 2,6 à 8,9 pendant que le couple de vissage tombe sous 5,0 N·m
- ☐ `01_de/sql/diagnostic_usine.sql` écrit, lancé d'un trait, et versionné
- ☐ L'heure résolue : 0 couple d'étiquette et d'instant en double, une fois le recul de l'horloge pris en compte
- ☐ L'argent écrit : `argent.usine_horaire` 240 240 lignes, `argent.production_horaire` 13 104, `argent.energie_horaire` 37 223
- ☐ Le lac argent : 1,1 Mo pour l'horaire, 52 Ko pour la production, 816 Ko pour l'énergie
- ☐ TimescaleDB chargée : 671 057 morceaux, 55 étiquettes, 27 tranches d'hypertable, 59 Mo
- ☐ L'agrégat continu : 239 816 lignes, et 0 écart avec le rééchantillonnage de DuckDB

Avant de commencer — la base de séries temporelles

© 2026 iAlexMG. Tous droits réservés.

Ce cours allume un service de plus, le dernier de la plateforme avant le chemin chaud du Cours 09. TimescaleDB est une extension de PostgreSQL : le même serveur, le même SQL, avec des tables découpées par tranche de temps. Son image pèse 2,17 Go et ne se tire qu'une fois.

Le service ne sert qu'à partir de l'étape 13 : si vous faites ce labo en deux séances, ce bloc peut attendre le début de la seconde, et c'est le plus simple, puisqu'il monte un dossier que l'étape 10 remplit.

La plateforme reçue au Cours 04 le décrit déjà. Ouvrez son `.env`, dans `Projet_Boreal\plateforme`, et donnez-lui le mot de passe que le modèle attend :

```
SERIES_MOT_DE_PASSE=choisissez-en-un
```

Le service monte en lecture seule `lac/argent/usine`, le dossier que l'étape 10 remplira. Créez-le avant de démarrer, sinon Docker le fera à votre place et le `mkdir` de l'étape 10 répondra qu'il existe déjà. Dans l'Anaconda Prompt, depuis `Projet_Boreal` :

```
mkdir plateforme\lac\argent\usine
cd plateforme
wsl
```

Dans Ubuntu, depuis `plateforme` :

```
docker compose up -d series
docker compose ps series
```

```
NAME            IMAGE                                STATUS
boreal_series   timescale/timescaledb:2.30.1-pg17     Up 12 seconds
```

Le service écoute sur le port 5434 : le 5433 est à la base de l'ERP, et le 5432 reste libre pour un PostgreSQL installé sur le poste. Vérifiez que l'extension est bien là :

```
docker compose exec series psql -U postgres -d usine -c "SELECT extversion FROM pg_extension WHERE extname = 'timescaledb';"
```

```
 extversion
------------
 2.30.1
```

Gardez ce terminal Ubuntu ouvert : sous Windows 10, WSL arrête Ubuntu et ses conteneurs quelques secondes après la fermeture du dernier terminal.

Étape 1 — Le référentiel : de l'étiquette à l'équipement

Théorie : `Theorie_08_DE.md`, sections 1, 2, 5 et 6.

L'usine ne vous donne pas d'accès à son réseau de contrôle, et c'est normal : la section 6 dit pourquoi. Elle dépose des fichiers. Commencez par ceux qui décrivent le terrain, avant ceux qui le mesurent.

© 2026 iAlexMG. Tous droits réservés.

Ouvrez la ligne de commande DuckDB depuis `Data_Domain\Projet_Boreal` :

```
duckdb
```

Les actifs d'abord, avec leur hiérarchie :

```
-- Attendu : 23 lignes, dont 6 zones et 17 équipements.
SELECT type_actif, count(*) AS n
FROM read_csv('sources/usine/referentiel/actifs.csv')
GROUP BY type_actif
ORDER BY n DESC, type_actif;
```

Une zone n'a pas de parent, un équipement porte le sien. Les trois zones des lignes d'assemblage portent en plus le poste de travail de l'ERP qui leur correspond, et c'est le seul pont entre l'usine et les commandes :

```
-- Attendu : 3 lignes, ASM-MN, ASM-MM et ASM-HR.
SELECT actif_id, nom, poste_travail_erp
FROM read_csv('sources/usine/referentiel/actifs.csv')
WHERE poste_travail_erp IS NOT NULL
ORDER BY actif_id;
```

Les étiquettes ensuite. Chacune porte sa bande morte, et c'est elle qui décide du nombre de points que l'historien gardera :

```
-- Attendu : 58 étiquettes, réparties UTL 18, LHR 11, LMN 11, PEI 9, LMM 7, ESS 2.
SELECT SPLIT_PART(etiquette, '.', 1) AS zone, count(*) AS etiquettes
FROM read_csv('sources/usine/referentiel/etiquettes.csv')
GROUP BY zone
ORDER BY zone;
```

Retenez la répartition par zone : elle servira à l'étape 6 pour reconnaître une coupure de communication au seul nombre de points perdus.

Une étiquette se rattache à son équipement par `actif_id`, et les deux conventions de nommage se croisent ici, la première à points et la seconde à tiret :

```
-- Attendu : 11 lignes, les points du compresseur 2 et de la ligne des motoneiges.
SELECT e.etiquette, e.grandeur, e.unite, e.bande_morte, a.nom
FROM read_csv('sources/usine/referentiel/etiquettes.csv') e
JOIN read_csv('sources/usine/referentiel/actifs.csv') a ON a.actif_id = e.actif_id
WHERE e.actif_id IN ('UTL-CMP2', 'LMN-VIS', 'LMN-CNV')
ORDER BY e.etiquette;
```

Étape 2 — La GMAO, et ses dates qui n'en sont pas

Théorie : `Theorie_08_DE.md`, section 7.

© 2026 iAlexMG. Tous droits réservés.

Les bons de travail disent ce que la maintenance a fait, et quand. Comptez-les d'abord :

```
-- Attendu : 825 bons, 444 préventifs, 381 correctifs.
SELECT type_bt, count(*) AS n
FROM read_csv('sources/usine/gmao/bons_travail.csv')
GROUP BY type_bt
ORDER BY type_bt;

-- Attendu : 4 bons encore ouverts au 2026-03-31.
SELECT count(*) AS ouverts
FROM read_csv('sources/usine/gmao/bons_travail.csv')
WHERE ferme_le IS NULL;
```

Maintenant, la question qui compte : de quand date le plus ancien bon. La réponse évidente est fausse, et il faut la voir de ses yeux :

```
-- Attendu : 01/02/2024 11:00 et 31/12/2024 08:02, sur des bons qui vont de 2022 à 2026.
SELECT min(ouvert_le) AS premier, max(ouvert_le) AS dernier,
       typeof(any_value(ouvert_le)) AS type_lu
FROM read_csv('sources/usine/gmao/bons_travail.csv');
```

DuckDB a lu `ouvert_le` en `VARCHAR`, parce que `JJ/MM/AAAA HH:MM` ne ressemble à aucun format de date reconnu. Le minimum et le maximum sont donc alphabétiques : `01/...` précède `31/...`, et l'année, qui vient en dernier, ne pèse rien dans le tri. Déclarez le format, et tout rentre dans l'ordre :

```
-- Attendu : 2022-01-06 15:50:00 et 2026-03-31 16:18:00.
SELECT min(STRPTIME(ouvert_le, '%d/%m/%Y %H:%M')) AS premier,
       max(STRPTIME(ouvert_le, '%d/%m/%Y %H:%M')) AS dernier
FROM read_csv('sources/usine/gmao/bons_travail.csv');
```

Piège — une date en texte ne se trie pas, elle s'aligne. Le tri alphabétique d'une date au format jour-mois-année classe par jour du mois. Aucune erreur n'est levée, aucun message n'apparaît, et un tableau de bord bâti là-dessus donne des chiffres faux avec un aplomb parfait. C'est le même défaut que les dates `DATS` de l'ERP au Cours 03, sous une autre forme : la `RAW` garde le texte, et c'est au consommateur de le qualifier.

Étape 3 — Le compteur électrique : un horodatage qui dit la fin

Théorie : `Theorie_08_DE.md`, section 7.

Le compteur principal livre un relevé aux quinze minutes, un fichier par mois, depuis 2022 :

```
-- Attendu : 148 891 relevés, 1 compteur, du 2022-01-01 00:15:00-05 au 2026-03-31 23:45:00-04.
SELECT count(*) AS releves, count(DISTINCT no_compteur) AS compteurs,
       min(fin_intervalle) AS premier, max(fin_intervalle) AS dernier
FROM read_csv('sources/usine/compteur/releves_*.csv');
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- Attendu : 148 644 relevés réels et 247 estimés.
SELECT qualite, count(*) AS n
FROM read_csv('sources/usine/compteur/releves_*.csv')
GROUP BY qualite
ORDER BY qualite;
```

Les 247 relevés estimés sont une valeur reconstruite par le système de gestion de l'énergie après une perte de communication. Ils ne sont pas faux, mais ils ne sont pas mesurés, et un calcul de facture ne les traite pas comme les autres.

Le décalage à UTC est écrit dans la colonne, ce qui est la bonne pratique. Vérifiez maintenant ce qu'il coûte de l'ignorer. `AT TIME ZONE` lit l'instant dans le fuseau qu'on nomme, et c'est lui qui décide du jour ; l'heure de mur, l'instant et le fuseau de session sont au Cours 02 de la formation SQL avancé, section 6, et la journée vue d'un fuseau à son Cours 09, section 1 :

```sql
-- Attendu : 27 116 relevés, soit 18 % du total, ne tombent pas le même jour selon le fuseau de lecture.
SELECT count(*) AS releves_qui_changent_de_jour
FROM read_csv('sources/usine/compteur/releves_*.csv')
WHERE CAST(fin_intervalle AT TIME ZONE 'America/Toronto' AS DATE)
   <> CAST(fin_intervalle AT TIME ZONE 'UTC' AS DATE);
```

Puis la convention de bornage. L'horodatage marque la fin de l'intervalle, donc le relevé de minuit du 1er janvier appartient à l'année précédente pour qui compte par le début :

```sql
-- Attendu : 4 995 612 kWh par la fin de l'intervalle, 4 995 732 kWh par le début. Même donnée.
SELECT ROUND(SUM(energie_kwh) FILTER (WHERE YEAR(fin_intervalle) = 2022)) AS par_la_fin,
       ROUND(SUM(energie_kwh) FILTER (WHERE YEAR(fin_intervalle - INTERVAL 15 MINUTE) = 2022)) AS par_le_debut
FROM read_csv('sources/usine/compteur/releves_*.csv');
```

Cent vingt kilowattheures d'écart sur cinq millions, c'est peu, et c'est bien le problème : aucun contrôle de vraisemblance ne le verra, et deux rapports bâtis sur deux conventions ne se réconcilieront jamais. La règle est d'écrire la convention à côté du chiffre, une fois pour toutes.

Une dernière lecture, celle qu'un responsable de l'énergie demande en premier :

```sql
-- Attendu : 1 906 kW, sur l'intervalle qui finit le 2022-02-25 07:15:00-05.
SELECT ROUND(MAX(energie_kwh) * 4) AS pointe_kw,
       arg_max(fin_intervalle, energie_kwh) AS intervalle
FROM read_csv('sources/usine/compteur/releves_*.csv');
```

L'énergie d'un quart d'heure se multiplie par quatre pour donner une puissance moyenne en kilowatts. C'est cette puissance-là que le tarif facture, et c'est pour elle que la seconde séance calculera une série horaire. `arg_max(fin_intervalle, energie_kwh)` rend l'intervalle de la ligne où l'énergie est la plus forte : la formation SQL avancé l'enseigne sous son autre nom, `MAX_BY`, au Cours 09, section 5.

Étape 4 — L'historien au bronze : ce qui n'est pas une mesure

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_08_DE.md`, section 3.

Le bronze de l'historien est en place depuis le Cours 02, et le Cours 07 l'a laissé à 182 jours. Vérifiez-le avant de compter quoi que ce soit, par une vue : elle nomme une requête sans en stocker le résultat, et chaque lecture la relance (Cours 03 de la formation SQL avancé, section 7).

```
CREATE OR REPLACE VIEW historien AS
SELECT * FROM read_parquet('plateforme/lac/bronze/usine/historien/**/*.parquet',
                           hive_partitioning = true);

-- Attendu : 550 087 lignes, 58 étiquettes, 182 jours, du 2025-10-01 au 2026-03-31.
SELECT count(*) AS lignes, count(DISTINCT TagName) AS etiquettes,
       count(DISTINCT jour) AS jours, min(jour) AS premier, max(jour) AS dernier
FROM historien;
```

Si le compte diffère, le projet n'est pas dans l'état où le Cours 07 l'a laissé : reprenez son étape 8 avant d'aller plus loin.

La colonne `jour` vient du partitionnement du bronze : c'est le jour du fichier livré. La colonne `Timestamp` vient de l'historien. Les deux ne coïncident pas toujours, et c'est tout le sujet :

```
-- Attendu : 10 556 lignes, soit exactement 58 par jour sur 182 jours.
SELECT count(*) AS valeurs_de_depart
FROM historien
WHERE CAST(Timestamp AS DATE) <> jour;
```

Ces 10 556 lignes sont les valeurs de départ : la dernière valeur retenue la veille, recopiée en tête du fichier du lendemain pour que celui qui ne lit qu'un fichier sache d'où part chaque point. Elles existent aussi dans le fichier de la veille, à leur place. Qui charge tous les fichiers sans y penser les compte deux fois.

Posez donc une vue qui ne garde que les mesures du jour, et travaillez avec elle jusqu'à la fin du labo :

```
CREATE OR REPLACE VIEW points AS
SELECT * FROM historien WHERE CAST(Timestamp AS DATE) = jour;

-- Attendu : 539 531 points, soit 550 087 moins les 10 556 valeurs de départ.
SELECT count(*) AS points FROM points;
```

L'autre ligne technique est l'archivage forcé de minuit. L'historien écrit la valeur de chaque étiquette à `00:00:00`, qu'elle ait bougé ou non :

```
-- Attendu : 10 525 points, et non 10 556.
SELECT count(*) AS archivage_de_minuit
FROM points
WHERE Timestamp = CAST(jour AS TIMESTAMP);
```

Trente et un manquent. Cherchez les jours fautifs plutôt que de conclure trop vite :

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Attendu : 4 jours, le 2025-10-28 (47), le 2025-12-03 (56), le 2025-12-20 (51), le 2026-01-29 (47).
SELECT jour, count(*) AS points_a_minuit
FROM points
WHERE Timestamp = CAST(jour AS TIMESTAMP)
GROUP BY jour
HAVING count(*) <> 58
ORDER BY jour;
```

Les nombres manquants, onze, deux, sept et onze, sont des tailles de zones : une coupure de communication tenait au moment de l'archivage. L'étape 6 y reviendra par l'autre bout, celui de la qualité.

Étape 5 — Les deux jours où l'horloge ment

Théorie : `Theorie_08_DE.md`, section 7.

L'historien écrit une heure locale sans fuseau. Sur 182 jours, la fenêtre couvre les deux changements d'heure, et ils ne se ressemblent pas.

Le premier ajoute une heure au calendrier :

```sql
-- Attendu : 49 paires, toutes le 2025-11-02.
SELECT jour, count(*) AS paires_en_double
FROM (
  SELECT jour, TagName, Timestamp
  FROM points
  GROUP BY jour, TagName, Timestamp
  HAVING count(*) > 1
)
GROUP BY jour
ORDER BY jour;
```

Regardez-en une de près, pour comprendre ce que vous tenez :

```sql
-- Attendu : à 01:14 comme à 01:16, deux lignes de même étiquette et de même horodatage, portant
-- deux valeurs différentes : 18,22 et 19,38 à 01:14, 18,64 et 18,92 à 01:16.
SELECT TagName, Timestamp, Value, Quality
FROM points
WHERE jour = DATE '2025-11-02' AND TagName = 'ESS.BANC.TEMP_REFROIDISSEMENT'
  AND Timestamp BETWEEN TIMESTAMP '2025-11-02 01:14:00' AND TIMESTAMP '2025-11-02 01:20:00'
ORDER BY Timestamp, Value;
```

Ce ne sont pas des doublons de livraison, comme l'extrait de mars 2023 du Cours 03. Ce sont deux mesures réelles, prises à une heure d'intervalle, que l'horloge a rendues indistinguables : la première à 01:14 heure avancée, la seconde à 01:14 heure normale. Les effacer perdrait une heure de production ; les garder casse toute clé qui suppose un horodatage unique par étiquette.

© 2026 iAlexMG. Tous droits réservés.

Le second changement, lui, retire une heure :

```
-- Attendu : une seule ligne, 79 points le 2026-03-01. Le 2026-03-08 n'apparaît pas du tout, et
-- c'est la réponse : un groupe sans ligne ne rend pas un zéro, il ne rend rien.
SELECT jour, count(*) AS points_entre_2h_et_3h
FROM points
WHERE jour IN (DATE '2026-03-01', DATE '2026-03-08')
  AND HOUR(Timestamp) = 2
GROUP BY jour
ORDER BY jour;
```

Le trou est parfait et parfaitement normal : cette heure-là n'a pas existé. Un contrôle de complétude qui exige 24 heures par jour lèvera une fausse alerte tous les ans, au printemps, et une vraie panne passera à côté pendant qu'on l'ignore.

À retenir — les deux transitions se corrigent au même endroit et de la même façon : en reconstruisant l'instant absolu, par le fuseau `America/Toronto`, au moment où on nettoie la série. C'est le travail de la seconde séance. Ce qui se décide aujourd'hui, c'est de ne jamais écrire de clé sur `(TagName, Timestamp)` tant que cet horodatage est local.

Étape 6 — La qualité, les coupures, et le capteur qui ne bouge plus

Théorie : `Theorie_08_DE.md`, sections 4 et 8.

Comptez la qualité, et regardez ce que porte chaque code :

```
-- Attendu : 192 bonne 538 938, 64 douteuse 64, 0 mauvaise 529 ; et aucune valeur sous le code 0.
SELECT Quality, count(*) AS points, count(Value) AS avec_valeur,
       count(DISTINCT TagName) AS etiquettes, count(DISTINCT jour) AS jours
FROM points
GROUP BY Quality
ORDER BY Quality;
```

La colonne `avec_valeur` est la leçon de l'étape : `count(Value)` ne compte pas les valeurs absentes. Sous le code 0, l'historien n'écrit rien, et la ligne n'existe que pour dire qu'à cet instant, il n'y avait pas de mesure.

Les coupures se lisent par zone :

```
-- Attendu : cinq coupures de 18 points, les 18 étiquettes des utilités, une seule zone chaque fois.
SELECT jour, min(Timestamp) AS instant, count(*) AS points_perdus,
       count(DISTINCT SPLIT_PART(TagName, '.', 1)) AS zones_touchees
FROM points
WHERE Quality = 0
GROUP BY jour, DATE_TRUNC('minute', Timestamp)
```


© 2026 iAlexMG. Tous droits réservés.

```
ORDER BY points_perdus DESC, instant
LIMIT 5;
```

Passez au capteur figé. La requête naïve cherche les plages où la valeur ne change pas : elle marque chaque changement en comparant la valeur à la précédente (LAG), puis numérote les plages par une somme cumulée de ces marques. C'est la technique des îlots, au Cours 09 de la formation SQL avancé, section 7, bâtie sur les fenêtres de son Cours 04, sections 4 et 5 :

```
-- Attendu : cinq points de la ligne des motomarines, tous à 0, de 768 à 790 heures, du
-- 2025-12-17 au 2026-01-19.
WITH marques AS (
  SELECT TagName, Timestamp, Value,
         CASE WHEN Value IS DISTINCT FROM LAG(Value) OVER (PARTITION BY TagName ORDER BY Timestamp)
              THEN 1 ELSE 0 END AS change
  FROM points WHERE Quality = 192
), plages AS (
  SELECT TagName, Value, Timestamp,
         SUM(change) OVER (PARTITION BY TagName ORDER BY Timestamp ROWS UNBOUNDED PRECEDING) AS plage
  FROM marques
)
SELECT TagName, Value, min(Timestamp) AS debut, max(Timestamp) AS fin,
       ROUND(DATE_DIFF('minute', min(Timestamp), max(Timestamp)) / 60.0, 1) AS heures
FROM plages
GROUP BY TagName, Value, plage
HAVING count(*) > 1
ORDER BY heures DESC
LIMIT 5;
```

Les cinq résultats annoncent plus de 760 heures de valeur constante, tous sur la ligne des motomarines, du 17 décembre au 19 janvier. Ce n'est pas un capteur mort : la ligne est arrêtée pendant cette période, parce que la saison des motoneiges occupe l'usine. La valeur est zéro, et c'est la bonne valeur.

Rejouez la même requête en écartant les valeurs nulles, et le vrai cas apparaît en tête :

```
-- Attendu : trois plages, dont la première seule est un capteur figé. PEI.CAB.HUMIDITE, 54,52 %, du
-- 2025-10-07 10:10 au 2025-10-09 12:00, 51 points, 49,8 heures ; puis la vibration du compresseur 1,
-- 24 heures ; puis un compteur, 15,6 heures.
WITH marques AS (
  SELECT TagName, Timestamp, Value,
         CASE WHEN Value IS DISTINCT FROM LAG(Value) OVER (PARTITION BY TagName ORDER BY Timestamp)
              THEN 1 ELSE 0 END AS change
  FROM points WHERE Quality = 192
), plages AS (
  SELECT TagName, Value, Timestamp,
         SUM(change) OVER (PARTITION BY TagName ORDER BY Timestamp ROWS UNBOUNDED PRECEDING) AS plage
```


© 2026 iAlexMG. Tous droits réservés.

```
  FROM marques
)
SELECT TagName, Value, min(Timestamp) AS debut, max(Timestamp) AS fin, count(*) AS points,
       ROUND(DATE_DIFF('minute', min(Timestamp), max(Timestamp)) / 60.0, 1) AS heures
FROM plages
WHERE Value <> 0
GROUP BY TagName, Value, plage
HAVING count(*) > 1 AND DATE_DIFF('hour', min(Timestamp), max(Timestamp)) >= 6
ORDER BY heures DESC
LIMIT 3;
```

Cinquante et un points, tous de qualité bonne, tous à 54,52 %, pendant que la cabine de peinture travaille. Aucun contrôle de plage, de type ou de qualité ne l'aurait attrapé.

Les deux autres lignes ne sont pas des capteurs figés, et il vaut la peine de voir pourquoi avant de les classer. La vibration du compresseur d'air 1 tient 0,01 pendant vingt-quatre heures au tournant de l'année, et le filtre sur le zéro ne l'a pas écartée, parce que sa valeur d'arrêt n'est pas zéro. Regardez les autres points de la même machine sur la même plage :

```
-- Attendu : quatre étiquettes. La marche à 0, avec un seul point, celui de l'archivage de minuit ;
-- la puissance à 0 sur 25 points ; la vibration à 0,01 sur 25 points ; et la température de
-- refoulement qui redescend de 17,55 à 15,74 °C.
SELECT TagName, count(*) AS points, ROUND(MIN(Value), 2) AS mini, ROUND(MAX(Value), 2) AS maxi
FROM points
WHERE SPLIT_PART(TagName, '.', 2) = 'CMP1'
  AND Timestamp BETWEEN TIMESTAMP '2025-12-31 07:00:00' AND TIMESTAMP '2026-01-01 07:00:00'
GROUP BY TagName
ORDER BY TagName;
```

Le compresseur est arrêté, sa puissance est nulle, et sa carcasse refroidit : 0,01 est ce que son capteur de vibration écrit dans ce cas. C'est le congé des Fêtes, pas une panne. La troisième ligne se règle de la même façon, un compteur de convoyeur qui ne bouge pas la nuit ne compte rien. La leçon tient en une phrase : sur une valeur constante, c'est l'état de marche qui tranche, jamais la valeur seule.

Notez où se pose le filtre : dans le `WHERE` de la requête finale, après le découpage en plages. Déplacez-le dans le premier bloc, avec la qualité, et le résultat devient absurde : les périodes de marche, débarrassées des arrêts qui les séparaient, se soudent en une seule plage, et la requête annonce trois états de marche figés pendant 4 344 heures, soit les 182 jours du jeu. Essayez-le une fois, c'est instructif.

Piège — écarter le zéro n'est pas une règle générale. Ici, cela dégage la liste parce que les grandeurs figées à zéro sont des équipements arrêtés, mais vous venez de voir la limite du procédé : le compresseur 1 s'arrête à 0,01 et traverse le filtre. Sur une mesure qui vaut légitimement zéro en fonctionnement, une pression différentielle par exemple, ce même filtre cacherait le vrai cas. La parade robuste est de croiser avec l'état de marche de l'équipement, ce que l'étape 7 fait pour le compresseur.

© 2026 iAlexMG. Tous droits réservés.

Étape 7 — L'incident du compresseur 2, lu dans les points

Théorie : `Theorie_08_DE.md`, section 3.

La GMAO connaît deux bons pour le compresseur d'air 2 en octobre 2025 :

```
-- Attendu : BT-2025-00739 le 22/10 (rien constaté) et BT-2025-00742 le 28/10 (soupape et roulement).
SELECT no_bt, type_bt, priorite, ouvert_le, ferme_le, description
FROM read_csv('sources/usine/gmao/bons_travail.csv')
WHERE actif_id = 'UTL-CMP2' AND ouvert_le LIKE '%/10/2025%'
ORDER BY STRPTIME(ouvert_le, '%d/%m/%Y %H:%M');
```

Vu de la maintenance, l'histoire commence le 22 octobre par un bruit signalé et se règle le 28. Vue des points, elle commence treize jours plus tôt. Une moyenne quotidienne le montre mal :

```
-- Attendu : des jours à 0,02 (le compresseur est arrêté) mêlés aux jours de marche.
SELECT jour, ROUND(AVG(Value), 2) AS vibration_moyenne, count(*) AS points
FROM points
WHERE TagName = 'UTL.CMP2.VIBRATION' AND jour BETWEEN DATE '2025-10-13' AND DATE '2025-10-31'
GROUP BY jour
ORDER BY jour;
```

Les fins de semaine et les jours de réparation écrasent le signal. Il faut ne garder que les mesures prises pendant que l'équipement tourne, et l'état de marche n'est retenu qu'à ses changements : c'est une jointure as-of, qui rattache chaque mesure au dernier état connu.

```
-- Attendu : 13 jours, de 2,57 le 14 octobre à 8,93 le 28, puis 2,40 le 30 après réparation.
WITH vibration AS (
  SELECT Timestamp, Value FROM points WHERE TagName = 'UTL.CMP2.VIBRATION' AND Quality = 192
), marche AS (
  SELECT Timestamp, Value AS etat FROM points WHERE TagName = 'UTL.CMP2.MARCHE' AND Quality = 192
)
SELECT CAST(v.Timestamp AS DATE) AS jour, ROUND(AVG(v.Value), 2) AS vibration_en_marche,
       count(*) AS points
FROM vibration v ASOF JOIN marche m ON v.Timestamp >= m.Timestamp
WHERE m.etat = 1 AND CAST(v.Timestamp AS DATE) BETWEEN DATE '2025-10-13' AND DATE '2025-10-31'
GROUP BY jour
ORDER BY jour;
```

La dérive est nette : 2,57, 2,45, 2,59, 2,68 la première semaine, puis 3,39, 3,90, 4,49, 4,96, 5,67, et 8,06 le 27 octobre, la veille du déclenchement. Les jours d'arrêt ont disparu d'eux-mêmes.

Rappel — la jointure as-of appartient au Cours 09 de la formation SQL avancé, qui l'enseigne pour de bon avec les séries temporelles. Ici, on s'en sert pour ce qu'elle fait : pour chaque mesure, retrouver la dernière ligne d'une autre série dont l'horodatage lui est antérieur ou égal.

© 2026 iAlexMG. Tous droits réservés.

Le réseau d'air raconte la suite de l'histoire. La pression tient autour de 700 kPa, et se met à plonger quand les lignes tirent :

```
-- Attendu : le minimum en journée descend de 689 kPa le 14 octobre à 517 le 24, sans passer sous 500.
SELECT jour, ROUND(MIN(Value)) AS pression_min, ROUND(AVG(Value)) AS pression_moyenne,
       count(*) AS points
FROM points
WHERE TagName = 'UTL.AIR.PRESSION' AND Quality = 192
  AND jour BETWEEN DATE '2025-10-14' AND DATE '2025-10-28'
  AND HOUR(Timestamp) BETWEEN 7 AND 14
GROUP BY jour
HAVING MIN(Value) > 0
ORDER BY jour;
```

Deux choses à voir dans ce résultat. Le minimum ne descend jamais sous 500 kPa, le seuil qui ferait démarrer le compresseur d'appoint : voilà pourquoi personne n'a été alerté. Et le nombre de points passe de douze à plus de cent cinquante dans la même fenêtre de journée, parce qu'une grandeur qui oscille franchit sa bande morte bien plus souvent.

Le volume de points est donc un signal à lui seul, et il se mesure sans rien connaître du métier :

```
-- Attendu : 89 points les 2 et 6 octobre, 198 le 20, 265 le 24, 244 le 27.
SELECT jour, count(*) AS points_pression
FROM points
WHERE TagName = 'UTL.AIR.PRESSION'
  AND jour IN (DATE '2025-10-02', DATE '2025-10-06', DATE '2025-10-20',
               DATE '2025-10-24', DATE '2025-10-27')
GROUP BY jour
ORDER BY jour;
```

Cherchez la même chose au grain de la zone, et vous ne verrez rien : les utilités portent 1 447 points le 28 octobre, au milieu de leur fourchette ordinaire. Un indicateur de volume ne vaut qu'à l'étiquette.

Le bout de la chaîne est au poste de vissage de la ligne des motoneiges, en bout de réseau d'air :

```
-- Attendu : couple moyen 5,61 et minimum 5,42 le 15 octobre ; 4,33 et 2,68 le 27.
SELECT jour, ROUND(AVG(Value), 2) AS couple_moyen, ROUND(MIN(Value), 2) AS couple_min
FROM points
WHERE TagName = 'LMN.VIS.COUPLE' AND Quality = 192 AND Value > 0
  AND jour IN (DATE '2025-10-15', DATE '2025-10-22', DATE '2025-10-27', DATE '2025-10-31')
GROUP BY jour
ORDER BY jour;
```

Le couple minimal de serrage est de 5,0 N·m. Le 27 octobre, des colliers sont serrés à 2,68. Ces véhicules-là sont partis chez les concessionnaires, et le Cours 11 reviendra les chercher par leur numéro de série, quand il s'agira d'écrire un contrat qui aurait attrapé la dérive.

© 2026 iAlexMG. Tous droits réservés.

Étape 8 — Rassembler le diagnostic, et le versionner

Les neuf contrôles de ce labo valent d'être rejoués à chaque livraison de l'usine, et pas seulement aujourd'hui. Rassemblez-les dans un script.

Dans l'Anaconda Prompt, depuis `Data_Domain\Projet_Boreal\01_de` :

```
notepad sql\diagnostic_usine.sql
```

Ce code ne se tape pas dans la ligne de commande DuckDB : c'est un fichier du dépôt, que le Bloc-notes crée et que DuckDB lira ensuite d'un trait. Le contenu complet est dans `02_Boreal/Solution/01_de/sql/diagnostic_usine.sql` ; il reprend, dans l'ordre, les comptes du référentiel, ceux de la GMAO, ceux du compteur, les lignes techniques de l'historien, les deux changements d'heure, la qualité et le capteur figé, chacun avec son attendu en commentaire.

Au moment d'enregistrer, choisissez « Tous les fichiers » dans la liste des types, sans quoi le Bloc-notes ajoute `.txt` au nom.

Lancez-le d'un trait, depuis `Projet_Boreal` :

```
duckdb -f 01_de\sql\diagnostic_usine.sql
```

Chaque requête doit rendre l'attendu écrit au-dessus d'elle. Un écart veut dire que les sources ne sont pas celles du cours : reprenez l'étape 8 du Labo 07.

Versionnez, depuis `01_de` :

```
git add sql/diagnostic_usine.sql
git commit -m "Cours 08 : diagnostic des sources de l'usine"
```

Étape 9 — Résoudre l'heure, et le prouver

Théorie : `Theorie_08_DE.md`, section 9.

Tout ce qui suit travaille en instants absolus. La conversion bute sur l'heure doublée du 2 novembre, et DuckDB tranche silencieusement en faveur de l'heure normale :

```
-- Attendu : 01:30:00-05 pour l'heure doublée, et 03:30:00-04 pour une heure qui n'existe pas.
SELECT TIMESTAMP '2025-11-02 01:30:00' AT TIME ZONE 'America/Toronto' AS heure_doublee,
       TIMESTAMP '2026-03-08 02:30:00' AT TIME ZONE 'America/Toronto' AS heure_absente;
```

La première occurrence, celle de l'heure avancée, serait donc datée d'une heure trop tard. Le fichier sait pourtant laquelle est laquelle : dans l'ordre de ses lignes, l'horodatage recule une seule fois, et ce recul est la bascule.

```
-- Attendu : une seule ligne, au rang 225, où 01:00 succède à 01:56.
SELECT file_row_number, Timestamp,
       LAG(Timestamp) OVER (ORDER BY file_row_number) AS precedent
FROM read_parquet('plateforme/lac/bronze/usine/historien/jour=2025-11-02/*.parquet',
                  file_row_number = true)
```


© 2026 iAlexMG. Tous droits réservés.

```
WHERE CAST(Timestamp AS DATE) = DATE '2025-11-02'
QUALIFY Timestamp < LAG(Timestamp) OVER (ORDER BY file_row_number);

-- Attendu : 71 points avant le recul, en heure avancée, et 70 après, en heure normale.
SELECT CASE WHEN file_row_number < 225 THEN 'heure avancee' ELSE 'heure normale' END AS cote,
       count(*) AS points
FROM read_parquet('plateforme/lac/bronze/usine/historien/jour=2025-11-02/*.parquet',
                  file_row_number = true)
WHERE CAST(Timestamp AS DATE) = DATE '2025-11-02' AND HOUR(Timestamp) = 1
GROUP BY cote ORDER BY cote;
```

Les 141 points de l'heure doublée se répartissent donc sans reste, et sans arbitraire.

Étape 10 — L'argent : découper, pondérer, écrire

Théorie : `Theorie_08_DE.md`, sections 9 et 10.

Le calcul complet tient dans un script. Créez-le, depuis l'Anaconda Prompt, dans `Projet_Boreal\01_de` :

```
notepad sql\argent_usine.sql
```

Le contenu complet est dans `02_Boreal/Solution/01_de/sql/argent_usine.sql`. Il enchaîne, dans cet ordre : le bronze lu avec le rang de chaque ligne ; les lignes du jour, valeurs de départ écartées ; le recul de l'horloge, jour par jour ; l'instant absolu ; la validité de chaque point jusqu'au suivant, le dernier fermé à la fin de la période ; la grille des heures ; le découpage de chaque validité aux frontières d'heure ; puis les trois tables.

Deux lignes méritent d'être lues avant de lancer. Celle qui ferme le dernier point, sans quoi la dernière journée de chaque étiquette disparaît. `TIME_BUCKET` y rabat un instant sur le début de son heure ; les périodes de largeur libre, et l'origine qu'il faut savoir nommer, sont au Cours 09 de la formation SQL avancé, section 5 :

```
COALESCE(LEAD(instant) OVER (PARTITION BY etiquette ORDER BY instant),
         (SELECT TIME_BUCKET(INTERVAL 1 HOUR, max(instant)) + INTERVAL 1 HOUR
          FROM points_datees)) AS fin
```

Et celle qui pondère, qui est le cœur du cours :

```
SUM(valeur * secondes) FILTER (WHERE valeur IS NOT NULL)
  / NULLIF(SUM(secondes) FILTER (WHERE valeur IS NOT NULL), 0) AS valeur
```

Le dossier du lac doit exister avant le lancement : `COPY … PARTITION_BY` ne crée pas ses dossiers parents, comme au Cours 02. Si vous avez déjà allumé le service `series`, il est là, et la commande vous le dira sans conséquence.

```
mkdir ..\plateforme\lac\argent\usine
```

Fermez la ligne de commande DuckDB, puis lancez le script depuis `Projet_Boreal` :

© 2026 iAlexMG. Tous droits réservés.

```
duckdb -f 01_de\sql\argent_usine.sql
```

lignes	etiquettes	heures
240240	55	4368

Les contrôles du script doivent tous tomber juste : 424 heures sans aucune mesure, 859 partiellement couvertes, 0 doublon, 0 valeur sans couverture, 13 104 lignes de production, 0 jour où la somme des productions s'écarte du dernier cumul, 37 223 heures d'énergie dont une seule incomplète.

Étape 11 — Ce que la pondération change, mesuré

Théorie : `Theorie_08_DE.md`, sections 9 et 10.

Relancez la ligne de commande DuckDB depuis `Projet_Boreal`, et comparez les deux moyennes sur l'heure du déclenchement :

```
ATTACH 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);

-- Attendu : six heures pleines à 3 600 secondes de couverture, de 8,796 à 6 h jusqu'à 0,013 à
-- 11 h, en passant par 6,669 à 10 h, l'heure du déclenchement.
SELECT heure, valeur AS moyenne_ponderee, points, couverture_s
FROM entrepot.argent.usine_horaire
WHERE etiquette = 'UTL.CMP2.VIBRATION'
  AND heure >= TIMESTAMPTZ '2025-10-28 06:00:00-04'
  AND heure <  TIMESTAMPTZ '2025-10-28 12:00:00-04'
ORDER BY heure;
```

Cette table ne garde que la moyenne juste, et elle ne dit donc pas ce qu'elle évite. Le fichier des morceaux, que le script vient d'écrire, porte la durée de chacun : il permet de calculer les deux côte à côte.

```
-- Attendu : pendant la marche franche, les deux moyennes se tiennent à quelques millièmes, 8,796
-- contre 8,787 à 6 h. À 10 h, 6,669 pondérée contre 8,040 simple, sur neuf morceaux.
SELECT heure,
       ROUND(SUM(valeur * secondes) / SUM(secondes), 3) AS moyenne_ponderee,
       ROUND(AVG(valeur), 3) AS moyenne_simple,
       count(*) AS morceaux
FROM read_csv('plateforme/lac/argent/usine/morceaux_horaires.csv')
WHERE etiquette = 'UTL.CMP2.VIBRATION'
  AND heure >= TIMESTAMPTZ '2025-10-28 06:00:00-04'
  AND heure <  TIMESTAMPTZ '2025-10-28 12:00:00-04'
GROUP BY heure
ORDER BY heure;
```


© 2026 iAlexMG. Tous droits réservés.

Le compresseur a déclenché à 10 h 44 : la valeur est tombée à zéro pour le dernier quart de l'heure. La moyenne simple compte cet arrêt pour une ligne parmi neuf, la pondérée pour le temps qu'il a duré, et l'écart est de 1,371. Les cinq autres heures montrent l'autre moitié de la leçon : quand la grandeur est stable, les deux méthodes se rejoignent. C'est pour cela que l'écart se mesure sur toute la série plutôt que sur un exemple bien choisi, ce que l'étape 14 fera.

Vérifiez ensuite le contrôle des compteurs, celui qui prouve que la différence de cumuls est juste :

```
-- Attendu : 24 véhicules produits et 24 au dernier cumul, chaque jour ouvrable.
SELECT CAST(heure AT TIME ZONE 'America/Toronto' AS DATE) AS jour,
       SUM(production) AS produit, max(cumul) AS dernier_cumul
FROM entrepot.argent.production_horaire
WHERE etiquette = 'LMN.CNV.COMPTEUR'
  AND CAST(heure AT TIME ZONE 'America/Toronto' AS DATE) BETWEEN DATE '2025-10-20' AND DATE '2025-10-24'
GROUP BY jour ORDER BY jour;
```

Étape 12 — Ce que l'argent livre, et à qui

Théorie : `Theorie_08_DE.md`, section 10.

Trois tables sont maintenant dans le schéma `argent`, et les mêmes en Parquet dans le lac, partitionnées par mois. C'est le second produit que Data Engineering livre, après la couche `RAW` : Analytics Engineering en tirera l'étoile de l'usine, Data Science la maintenance prédictive et la prévision de la demande électrique.

```
-- Attendu : 240 240 lignes dans le lac comme dans l'entrepôt.
SELECT count(*) AS lignes_du_lac
FROM read_parquet('plateforme/lac/argent/usine/horaire/**/*.parquet');
```

Ouvrez `02_Boreal/Solution/LIVRAISON.md` et ajoutez-y la section des séries de l'usine : les trois tables, leur grain, leurs comptes attendus, et la commande qui les reconstruit. C'est ce document que les formations suivantes citeront.

Étape 13 — Charger la base de séries temporelles

Théorie : `Theorie_08_DE.md`, section 11.

Le quatrième fichier qu'a écrit le script de l'étape 10, `morceaux_horaires.csv`, celui que l'étape 11 vient de lire, est aussi ce que la base attend : les morceaux découpés, avec leur durée, en format qu'elle sait charger. Écrivez maintenant le script qui la monte, dans `Projet_Boreal\01_de` :

```
notepad sql\series_usine.sql
```

Le contenu complet est dans `02_Boreal/Solution/01_de/sql/series_usine.sql`. Il crée la table, la transforme en hypertable par tranches de sept jours, la charge, la passe au `VACUUM ANALYZE`, pose l'agrégat continu et sa politique de rafraîchissement.

Le conteneur ne voit que le dossier de l'argent, monté en lecture seule. Copiez-y le script, depuis Ubuntu, dans `plateforme` :

© 2026 iAlexMG. Tous droits réservés.

```
cp ../01_de/sql/series_usine.sql lac/argent/usine/
docker compose exec series psql -U postgres -d usine -f /argent/series_usine.sql
```

```
 create_hypertable
-------------------
 (1,t)
COPY 671057
VACUUM
 lignes | etiquettes
--------+------------
 671057 |         55
 tranches | taille
----------+--------
       27 | 59 MB
```

Vingt-sept tranches pour 182 jours : c'est le découpage par sept jours qui les produit. Une requête sur une journée n'en ouvrira qu'une. Notez le mot : les tranches sont ce que la base découpe pour se ranger, et les morceaux ce que le rééchantillonnage a découpé aux frontières d'heure. Ils n'ont rien à voir, et on en charge 671 057 dans 27 tranches.

Les 59 Mo se lisent après le `VACUUM ANALYZE`, et c'est pour cela qu'il suit le chargement. Sans lui, la même requête rend 63 Mo sur une base neuve : le chargement laisse des pages vides au bout de 18 des 27 tranches, que l'autovacuum ne rend que plus tard, à un moment que rien ne fixe.

Étape 14 — L'agrégat continu, et la preuve

Théorie : `Theorie_08_DE.md`, section 12.

La fin du script a créé l'agrégat continu et l'a rempli. Deux choses s'y lisent. D'abord, la moyenne simple ment, et on peut dire de combien :

```
 heures_ou_la_simple_ment | ecart_max
--------------------------+-----------
                   86521 |   9141.44
```

Plus d'une heure sur trois, et jusqu'à 9 141 unités d'écart sur le régime du banc d'essai. La requête arrondit l'écart à six décimales avant de le comparer au seuil : 398 heures tombent pile à un millième, et sans l'arrondi, le compte changerait d'un chargement à l'autre (théorie, section 9).

Ensuite, la moyenne pondérée tombe exactement sur celle de DuckDB, et c'est la preuve de fin du cours. Exportez l'agrégat, depuis Ubuntu, dans `plateforme` :

```
docker compose exec -T -e PGTZ=America/Toronto series psql -U postgres -d usine \
  -c "\copy (SELECT etiquette, seau AS heure, ROUND((somme_ponderee / secondes_totales)::numeric, 3) AS valeur, secondes_totales
AS couverture_s FROM usine_horaire ORDER BY etiquette, seau) TO STDOUT WITH (FORMAT csv, HEADER true)" \
  > lac/argent/usine/agregat_series.csv
```


© 2026 iAlexMG. Tous droits réservés.

Le fichier s'écrit par le terminal d'Ubuntu, et non par le conteneur : c'est pour cela qu'un montage en lecture seule suffit. Comparez ensuite, dans la ligne de commande DuckDB lancée depuis `Projet_Boreal` :

```
-- Attendu : 239 816 heures comparées, 0 écart de valeur, 0 écart de couverture.
SELECT count(*) AS heures_comparees,
       count(*) FILTER (WHERE ABS(s.valeur - a.valeur) > 0.001) AS ecarts_valeur,
       count(*) FILTER (WHERE s.couverture_s <> a.couverture_s) AS ecarts_couverture
FROM entrepot.argent.usine_horaire a
JOIN read_csv('plateforme/lac/argent/usine/agregat_series.csv') s
  ON s.etiquette = a.etiquette AND s.heure = a.heure
WHERE a.couverture_s > 0;
```

Deux moteurs que tout oppose, un entrepôt en colonnes sur des fichiers et une base transactionnelle à hypertables, rendent la même série au millième près. Ce n'est pas une coïncidence : c'est parce que le découpage et la durée ont été calculés une fois, à l'ingestion, et que les deux n'ont plus eu qu'à sommer.

Versionnez les deux scripts, depuis `01_de` :

```
git add sql/argent_usine.sql sql/series_usine.sql
git commit -m "Cours 08 : l'argent de l'usine, et la base de series temporelles"
```

Récapitulatif

Ce que vous avez mesuré	Le chiffre
Le référentiel	23 actifs, 6 zones, 17 équipements, 58 étiquettes
La GMAO	825 bons, 444 préventifs, 381 correctifs, 4 ouverts
Le compteur	148 891 relevés, 247 estimés, pointe à 1 906 kW
Les lignes techniques de l'historien	10 556 valeurs de départ, 10 525 archivages de minuit
Les changements d'heure	49 paires en double, une heure absente
La qualité	538 938 bonnes, 64 douteuses, 529 mauvaises sans valeur
Le capteur figé	49,8 heures à 54,52 %, et les arrêts d'équipement écartés, jusqu'à celui qui vaut 0,01
L'incident	vibration de 2,57 à 8,93 ; couple de vissage à 2,68 N·m
L'heure résolue	71 points en heure avancée, 70 en heure normale, 0 couple en double

© 2026 iAlexMG. Tous droits réservés.

Ce que vous avez mesuré	Le chiffre
L'argent	240 240 heures d'étiquettes, 13 104 de production, 37 223 d'énergie
La base de séries temporelles	671 057 morceaux, 27 tranches d'hypertable, 59 Mo
La preuve	239 816 heures, 0 écart entre l'agrégat continu et DuckDB

Ce que vous savez maintenant et que vous ne saviez pas : ces 550 087 lignes ne sont pas 550 087 mesures, tous les points ne portent pas de valeur, et 49 d'entre elles portent un horodatage qui ne désigne pas un instant unique. Vous en avez tiré une série horaire régulière, pondérée par le temps, qui porte sa propre couverture, et vous avez montré que deux moteurs opposés la recalculent à l'identique.

Ce que Data Engineering livre en sort : le schéma `argent` et le lac argent, que le Cours 12 reconstruira de bout en bout, et que le kit 02 fournira à Analytics Engineering pour l'étoile de l'usine et à Data Science pour la maintenance prédictive.

Dépannage

Symptôme	Cause	Ce qu'on fait
`IO Error: No files found that match the pattern` sur le bronze	la ligne de commande DuckDB n'a pas été lancée depuis `Projet_Boreal`	fermer, `cd` vers `Data_Domain\Projet_Boreal`, relancer
L'historien rend 275 436 lignes au lieu de 550 087	les sources sont celles de la génération par défaut, 92 jours	refaire l'étape 8 du Labo 07, avec `--jusqua 2026-03`
L'historien rend 2 407 796 lignes	les sources ont gardé `--historien complet` du Labo 07	même remède
`Binder Error: No function matches ASOF`	une version de DuckDB antérieure à 0.9	mettre à jour la ligne de commande, `winget upgrade DuckDB.cli`
Le fichier `diagnostic_usine.sql` reste introuvable pour DuckDB	le Bloc-notes l'a enregistré en `diagnostic_usine.sql.txt`	renommer, et choisir « Tous les fichiers » au prochain enregistrement
`Conversion Error` sur `STRPTIME`	le format donné ne correspond pas à la colonne	vérifier que la date est bien en `JJ/MM/AAAA HH:MM`, avec `SELECT ouvert_le FROM ... LIMIT 3`
`IO Error: Failed to create directory ... argent/usine/horaire`	`COPY … PARTITION_BY` ne crée pas ses dossiers parents	`mkdir ..\plateforme\lac\argent\usine`, puis relancer

© 2026 iAlexMG. Tous droits réservés.

Symptôme	Cause	Ce qu'on fait
`continuous aggregate view must include a valid time bucket function`	la colonne de sortie du seau porte le même nom que la colonne de temps	nommer la sortie autrement, `seau` par exemple
`could not open file "/argent/morceaux_horaires.csv"`	le fichier n'est pas dans le dossier monté, ou l'étape 10 n'a pas été relancée depuis l'ajout de l'export	vérifier `ls lac/argent/usine/` dans Ubuntu, puis relancer `duckdb -f 01_de\sql\argent_usine.sql`
`boreal_series` redémarre en boucle	le `.env` de la plateforme n'a pas de `SERIES_MOT_DE_PASSE`	l'ajouter, puis `docker compose up -d series`
Le conteneur a disparu entre deux commandes	le dernier terminal Ubuntu s'est fermé, et WSL a arrêté la distribution	rouvrir Ubuntu, `docker compose start series`, et garder un terminal ouvert

© 2026 iAlexMG. Tous droits réservés.